

SOMMAIRE

- 1. [La stack technique et pourquoi ces choix](#)
- 2. [L'architecture en couches](#)
- 3. [Le démarrage de l'application \(ce qui se passe au lancement\)](#)
- 4. [La sécurité : JWT, rôles, permissions](#)
- 5. [Le modèle de données complet \(57 tables\)](#)
- 6. [Module RH : employés, comptes, compétences, documents](#)
- 7. [Module Congés & Demandes \(le plus complexe côté RH\)](#)
- 8. [Module Calendrier & référentiels](#)
- 9. [Module Projets, tâches, équipes](#)
- 10. [Module Media Plan & Google Drive](#)
- 11. [Module Finance \(facturation, TVA, timbre fiscal\)](#)
- 12. [Module Paie tunisienne \(le morceau le plus technique\)](#)
- 13. [Module Monitoring & AgentDesktop](#)
- 14. [Notifications et emails](#)
- 15. [Les tâches planifiées \(schedulers\)](#)
- 16. [Gestion des erreurs](#)
- 17. [Configuration et environnements](#)
- 18. [Tests](#)
- 19. [Le module IA \(renvoi\)](#)

1. La stack technique et pourquoi ces choix

Fichier Backend/pom.xml

Dépendance	Version	À quoi ça sert
spring-boot-starter-parent	3.5.6	Gestion centralisée des versions
Java	17	LTS, records, switch expressions, text blocks
spring-boot-starter-web	—	API REST (Tomcat embarqué)
spring-boot-starter-data-jpa	—	ORM Hibernate + Repositories
spring-boot-starter-security	6.5.9	Filtrage des requêtes, BCrypt
spring-boot-starter-validation	—	Validation des DTOs (@NotNull , @Pattern)
spring-boot-starter-mail	—	Envoi SMTP
jjwt (api/impl/jackson)	0.12.5	Génération et vérification des JWT
postgresql	42.7.11	Driver JDBC
lombok	1.18.36	Suppression du boilerplate (@Data , @Builder)
langchain4j + langchain4j-open-ai	1.19.0	Framework IA (agents, outils, RAG)
springdoc-openapi	2.8.17	Swagger UI automatique
google-api-services-drive	v3	Création de dossiers Drive
testcontainers (postgresql)	—	Tests d'intégration sur un vrai Postgres

Point notable : les surcharges CVE

Le pom.xml surcharge délibérément certaines versions gérées par le BOM Spring Boot, avec les numéros de CVE en commentaire :

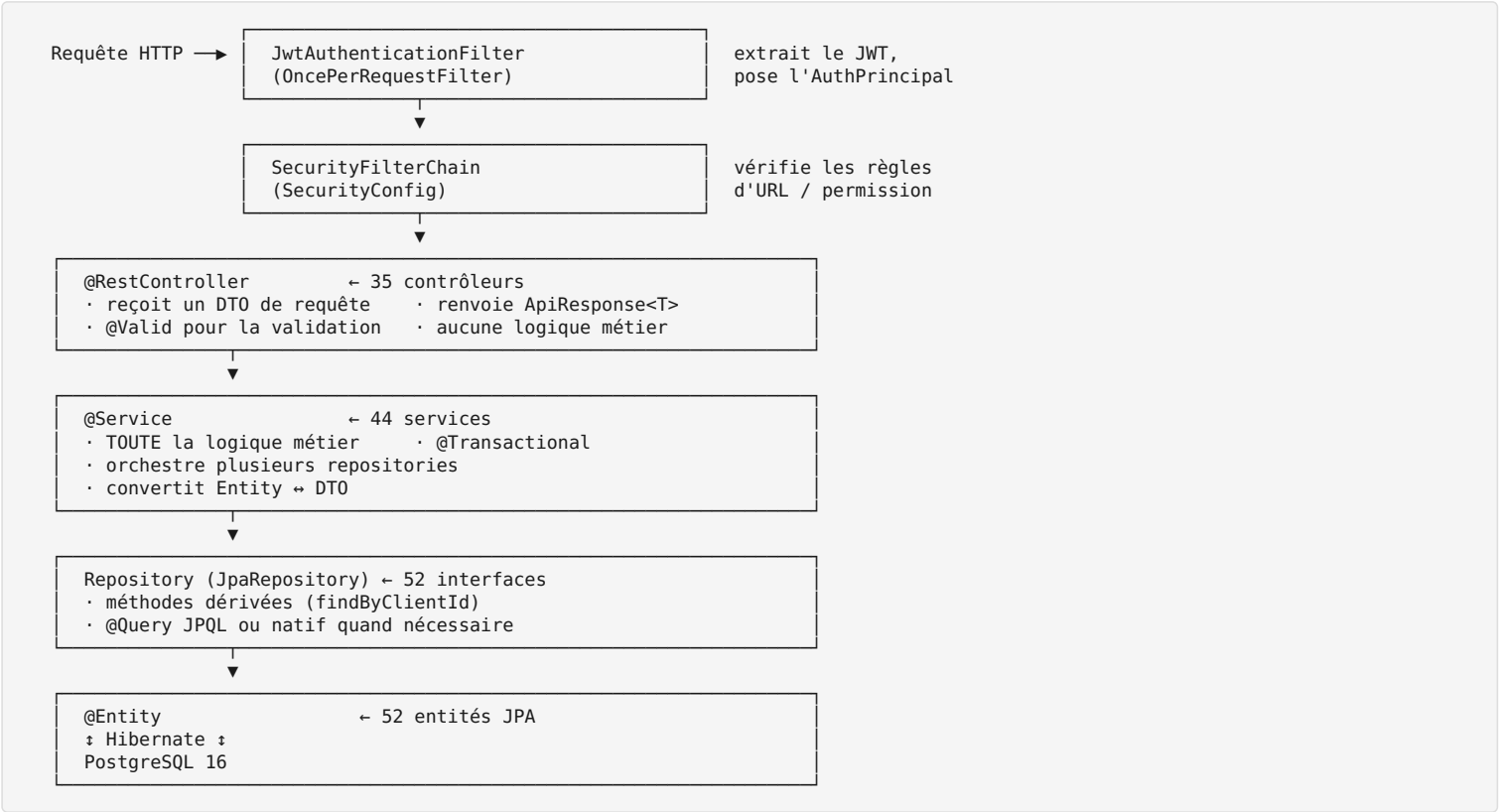
```
<tomcat.version>10.1.53</tomcat.version>           <!-- CVE-2025-24813, CVE-2024-50379... -->
<spring-security.version>6.5.9</spring-security.version> <!-- CVE-2024-38821... -->
<postgresql.version>42.7.11</postgresql.version>      <!-- CVE-2026-42198 -->
<json-smart.version>2.5.2</json-smart.version>         <!-- CVE-2024-57699 -->
```

À dire en soutenance : « J’ai fait un audit de vulnérabilités et j’ai surchargé les versions transitives concernées, en documentant chaque CVE dans le pom. »

Séparation tests unitaires / tests d’intégration

```
<!-- surefire n'exécute que les *Test → `mvn test` tourne sans Docker -->
<!-- failsafe exécute les *IT sur `mvn verify` → Testcontainers a besoin de Docker -->
```

2. L’architecture en couches



Pourquoi des DTOs et pas les entités directement ?

Trois raisons concrètes dans ce projet :

- 1. **Ne jamais exposer le passwordHash** — `CompteDTO` ne le contient pas.
- 2. **Éviter les boucles infinies de sérialisation** — `Employe` → `Compte` → `Employe` → ... Les entités ont `@ToString.Exclude` et `@JsonIgnore`, mais le DTO règle le problème proprement.
- 3. **Éviter le LazyInitializationException** — les relations sont en `FetchType.LAZY` ; la conversion en DTO se fait à l’intérieur de la transaction.

Les packages

```
com.antigone.rh
├─ RhApplication.java           ← point d'entrée (@SpringBootApplication)
├─ config/                     (6) configuration Spring : sécurité, CORS, init des données
├─ controller/                 (35) points d'entrée REST
└─ converter/                  (3) AttributeConverter JPA (JSON ↔ colonne texte)
```

dto/	(98)	objets de transport
entity/	(52)	tables
enums/	(20)	types fermés (statuts, types de congé...)
exception/	(2)	gestion centralisée des erreurs
repository/	(52)	accès données
scheduler/	(4)	tâches cron
security/	(4)	JWT
service/	(44)	logique métier
ai/		← MODULE IA COMPLET (voir 05-IA-EXPLIQUEE-SIMPLEMENT.md)

Point d’architecture à défendre : le module `ai` est un **sous-package autonome** avec ses propres entités, repositories, contrôleurs, sécurité et exceptions. Il consomme les repositories métier mais **le métier ne dépend jamais de l’IA**. Conséquence : si on retire la clé OpenAI, l’application RH fonctionne à 100 %, seuls les endpoints `/api/v1/**` répondent 503.

3. Le démarrage de l’application

Quatre classes agissent au lancement, dans cet ordre :

3.1 DatabaseUrlEnvironmentPostProcessor

S’exécute **avant** le contexte Spring. Render fournit une URL au format `postgresql://user:pass@host/db`, alors que le driver JDBC attend `jdbc:postgresql://host:5432/db`. Cette classe réécrit l’URL et extrait user/password. **Sans elle, le déploiement Render échoue au démarrage.**

3.2 SchemaCleanup

Nettoyages de schéma que `ddl-auto: update` ne sait pas faire (Hibernate ajoute des colonnes mais ne supprime jamais rien, ni ne modifie une contrainte existante).

3.3 DataInitializer (implements CommandLineRunner)

Amorce la base au premier démarrage — **et à chaque redémarrage**, de façon idempotente :

```
public void run(String... args) {
    initPermissionsAndRoles(); // 1. catalogue des permissions + rôle ADMIN
    initSystemParameters();    // 2. MAX AUTORISATION_MINUTES, SOLDE_CONGE_AN1...
    initTypeConge();           // 3. types de congé
    initGenres(); initTypesContrat(); initDepartements(); initPostes();
    initNiveauxHierarchiques(); initSitesEtablissement(); initTypesDemande();
    initFormatsMediaPlan(); initTypesMediaPlan(); initPlateformesMediaPlan();
    initCategoriesRevenu(); initCategoriesCharge(); // 4. référentiels
    initDefaultAdmin(); // 5. compte admin par défaut
    initBaremeIrpp(); // 6. barème fiscal tunisien
}
```

Le **rôle ADMIN reçoit automatiquement toutes les permissions**, y compris celles ajoutées après coup :

```
Role adminRole = roleRepository.findByNom("ADMIN").get();
adminRole.setPermissions(new HashSet<>(permissionRepository.findAll()));
```

Le **barème IRPP tunisien** par défaut (en vigueur depuis 2024) :

Tranche annuelle (TND)	Taux
0 – 5 000	0 %
5 000 – 10 000	15 %
10 000 – 20 000	25 %
20 000 – 30 000	30 %
30 000 – 40 000	33 %
40 000 – 50 000	36 %
50 000 – 70 000	38 %
> 70 000	40 %

3.4 GoogleDriveInitializationConfig

Charge la clé du compte de service Google. En production, la clé arrive en **Base64 dans une variable d'environnement** (`GOOGLE_DRIVE_SERVICE_ACCOUNT_KEY_B64`) et est décodée puis écrite dans un fichier temporaire — Render ne permet pas de déposer un fichier JSON secret.

4. La sécurité : JWT, rôles, permissions

4.1 Le cycle complet d'une authentification

```
1. POST /api/auth/login { username, password }
   |
2. CompteService : chargement du Compte par username
   → passwordEncoder.matches(password, compte.getPasswordHash()) [BCrypt]
   → si KO : 401
   |
3. Construction de l'AuthPrincipal :
   { principalType: "EMPLOYEE", accountId, employeId, username,
     roles: ["ADMIN"], permissions: ["VIEW_EMPLOYES", "VIEW_FINANCE", ...] }
   |
4. JwtService.generateToken(principal)
   → JWT signé HS256, valable 12 h (app.jwt.expiration-hours)
   → les rôles ET les permissions sont DANS le token
   |
5. Réponse LoginResponse { token, tokenExpiresAt, roles, permissions,
                          mustChangePassword, employeId, ... }
   |
6. À chaque requête suivante : header Authorization: Bearer <token>
   |
7. JwtAuthenticationFilter :
   → parse et vérifie la signature
   → convertit en GrantedAuthority :
       roles → "ROLE_ADMIN" (préfixe ROLE_ ajouté)
       permissions → "VIEW_FINANCE" (tel quel)
   → pose l'Authentication dans le SecurityContextHolder
```

Décision d'architecture importante : les permissions sont **dans le JWT**, pas rechargées en base à chaque requête. Avantage : zéro requête SQL pour autoriser. Inconvénient assumé : un changement de droits ne prend effet qu'à la reconnexion (au plus 12 h). C'est le compromis classique du **stateless**.

4.2 SecurityConfig — les règles

```
.csrf(disable) // API stateless, pas de cookie de session
.sessionManagement(STATELESS) // aucune session serveur
.dispatcherTypeMatchers(ASYNC, ERROR).permitAll() // indispensable pour le SSE (voir ci-dessous)

// PUBLIC
POST /api/auth/login, /client-login, /forgot-password, /reset-password
GET /api/agent/download, /api/clients/*/logo, /uploads/**
/v3/api-docs/**, /swagger-ui/**

// PROTÉGÉ
/api/admin/** → hasRole("ADMIN")
/api/comptes/** → ROLE_ADMIN ou VIEW_COMPTEs
/api/roles/** → ROLE_ADMIN ou VIEW_ROLES
/api/finance/** → ROLE_ADMIN ou VIEW_FINANCE
/api/v1/** (IA) → authenticated (le périmètre fin est géré dans les outils)
anyRequest() → authenticated
```

La ligne la plus subtile du fichier :

```
.dispatcherTypeMatchers(DispatcherType.ASYNC, DispatcherType.ERROR).permitAll()
```

Les réponses en streaming (`SseEmitter`) repassent une seconde fois dans la chaîne de filtres, en dispatch `ASYNC`, alors que le `SecurityContext` du thread initial a déjà été libéré. Sans cette règle, l'autorisation échoue sur une réponse déjà committée et le flux IA meurt sur une erreur servlet. **La décision d'accès a bien eu lieu sur le dispatch REQUEST.**

4.3 @EnableMethodSecurity

Sans cette annotation, **toutes les annotations @PreAuthorize du projet seraient ignorées silencieusement**. C'est un piège classique. Elle est indispensable au cloisonnement de l'assistant IA.

4.4 CORS

```
List<String> origines = Arrays.stream(frontendUrl.split(","))...
origines.add("https://*.onrender.com");
config.setAllowedOriginPatterns(origines);
```

Ajouter un frontend se fait **en configuration** (FRONTEND_URL), sans toucher au code.

4.5 Les deux types de principal

principalType	Qui	Champs remplis
EMPLOYEE	Un collaborateur	accountId , employeId , roles , permissions
CLIENT	Un client de l’agence (portail)	clientId , permissions limitées

Le client se connecte via POST /api/auth/client-login avec loginClient / passwordClient stockés sur l’entité Client , et la colonne clientPages liste les pages du portail qui lui sont ouvertes (MEDIA_PLANS , PROJETS , FICHIERS).

5. Le modèle de données complet

5.1 Domaine RH & identité (12 tables)

Table	Rôle	Points remarquables
employes	La personne	matricule unique, cin validé sur 8 chiffres, cnss, ribBancaire sur 20 chiffres, auto-relation manager → subordonnes (organigramme)
comptes	Identifiants	passwordHash BCrypt, mustChangePassword , resetToken + expiry
roles	Groupe de droits	N:N avec comptes et avec permissions
permissions	Droit atomique	26+ entrées
competences	Compétences d’un employé	
documents_employe	Pièces jointes RH	
equipes	Équipes (N:N avec employés)	
horaires_travail	Horaire type (heure début/fin, pause, jours)	
affectations_horaire	Quel employé sur quel horaire	
historique_statut	Journal des changements d’état	
notifications	Notifications in-app	
access_logs	Journal de connexion	

Détail des contraintes de validation sur Employe :

```
@Pattern(regexp = "[0-9]{8}$", message = "Le CIN doit contenir exactement 8 chiffres")
@Pattern(regexp = "[0-9]{1,12}(-[0-9]+)*$", message = "Le CNSS...")
@Pattern(regexp = "[0-9]{20}$", message = "Le RIB doit contenir exactement 20 chiffres")
```

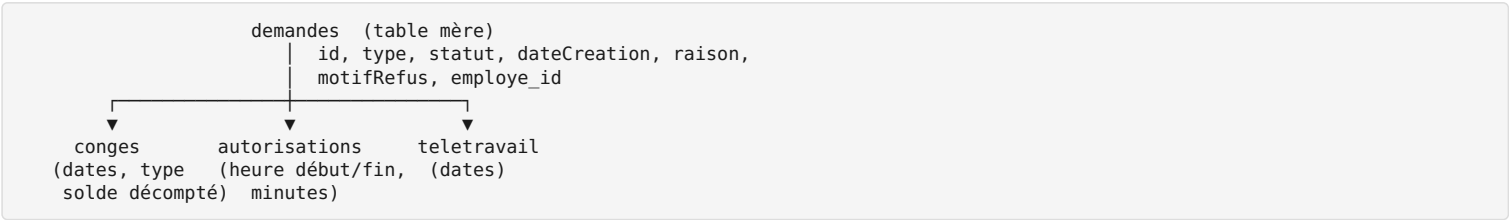
Ce sont les formats tunisiens réels.

Archivage plutôt que suppression :

```
private Boolean archived = false;
private LocalDateTime archivedAt;
```

On n’efface jamais un employé — les bulletins de paie, les congés et l’historique doivent rester consultables. La page « Archives » du front les affiche.

5.2 Domaine Demandes / Congés (7 tables) — héritage JOINED



C'est la stratégie JPA `@Inheritance(strategy = InheritanceType.JOINED)` : une table mère + une table fille par sous-type, jointes sur l' `id` .

Pourquoi ce choix ? Une demande de congé, une autorisation de sortie et une demande de télétravail partagent le **même cycle de validation** (créer → en attente → approuver/refuser → historiser) mais ont des champs différents. L'héritage JOINED factorise le cycle sans polluer les tables de colonnes nulles.

Tables associées : `validations` (qui valide, dans quel ordre, avec quelle décision), `historique_statut` , `periodes_bloquees` (périodes où les congés sont interdits).

5.3 Domaine Projets (9 tables)

Table	Rôle
<code>projets</code>	<code>statut</code> , <code>dateDebut/Fin</code> , <code>typeProjet</code> (DETERMINE/INDETERMINE), chef de projet et <code>projet_chefs</code> (plusieurs chefs)
<code>taches</code>	<code>statut</code> , <code>dateEcheance</code> , <code>urgente</code> , <code>archived</code> , <code>dureePrevueJours</code> , <code>driveLink</code>
<code>taches_obligatoires</code>	Tâches récurrentes imposées à une équipe ou un employé
<code>equipes</code>	+ table de jointure membres
<code>clients</code>	Fiche marque et identifiants du portail client
<code>contacts_client</code>	Contacts multiples par client
<code>calendrier_projet</code>	Événements projet (deadlines, réunions, tournages)
<code>reunions</code>	Réunions avec statut
<code>reactif_intern</code>	« Rectifs » : demandes de correction internes/externes

Le champ `Projet.isMediaPlanProject` : quand une ligne de media plan devient un tournage, le système génère automatiquement un projet dédié — d'où le lien `mediaPlanLigneId` .

5.4 Domaine Media Plan (4 tables)

`media_plans` est la table centrale du métier de l'agence :

```
datePublication, heure, format, type, titre,
texteSurVisuel, inspiration, autresElements, plateforme,
lienDrive,                               // dossier Drive du mois
etatPublication,                         // PAS_ENCORE / ...
rectifs, remarques,
// bloc SHOOTING
isShooting, shootingDescription, shootingLocalisation, shootingDate,
shootingTypeDeContenu, shootingStatus, shootingStatusReason,
statut,                                // EN_ATTENTE / ... (workflow de validation client)
client, createur, dateCreation
```

`media_plan_assignments` fait le lien **employé ↔ client** : c'est **cette table qui définit le périmètre** de ce qu'un social media manager peut voir — et c'est elle qu'interroge l'IA (`AiAccessScope.mediaPlanClientScope`).

`media_plan_comments` : fil de commentaires par ligne, y compris côté client.

5.5 Domaine Finance (17 tables)

Groupe	Tables
Facturation	<code>factures</code> , <code>paiements_facture</code> , <code>templates_facture</code> , <code>services_catalogue</code> , <code>compteurs_document</code> , <code>relances_client</code>
Paie	<code>bulletins_paie</code> , <code>parametres_paie</code> , <code>baremes_irpp</code> , <code>acomptes_salaire</code> , <code>declarations_cnss</code>
Charges & dettes	<code>charges_fixes</code> , <code>paiements_charge_fixe</code> , <code>charges_variatives</code> , <code>dettes</code> , <code>dette_paiements</code>
Revenus	<code>autres_revenus</code>

5.6 Domaine Monitoring (5 tables)

`pointages` , `heartbeats` , `presence_confirmations` , `rapports_inactivite` , `access_logs` .

5.7 Domaine IA (5 tables)

`ai_conversations` , `ai_messages` , `ai_document_chunks` , `ai_chat_memory` , `ai_tool_audit_log` . Détaillées dans `05-IA-EXPLIQUEE-SIMPLEMENT.md` .

5.8 Les convertisseurs JSON

Trois `AttributeConverter` stockent des structures complexes en colonne texte JSON :

Convertisseur	Utilisé par	Contenu
<code>LignesDocumentConverter</code>	<code>Facture.lignes</code>	Lignes de facture (désignation, quantité, PU)
<code>ElementsSalaireConverter</code>	<code>BulletinPaie.elements</code>	Primes, absences, ajustements variables
<code>TranchesIrppConverter</code>	<code>BaremeIrpp.tranches</code>	Tranches du barème fiscal

Pourquoi ? Ces listes n’ont **aucune valeur en dehors de leur parent** : on ne requête jamais « toutes les lignes de facture de mars ». Créer une table + une relation + une cascade de suppression pour ça serait de la complexité gratuite.

6. Module RH — employés, comptes, compétences

`EmployeService` (1052 lignes — le plus gros service)

Responsabilités : - CRUD employé avec génération automatique du **matricule** - Gestion de l’**organigramme** (relation manager/subordonnés, détection de cycles) - **Archivage** (`archived = true` + `archivedAt`) au lieu de suppression - Calcul du **solde de congé** selon l’ancienneté - Upload de la photo de profil et des documents - Historique des modifications (`HistoriqueEmployeDTO`) - Statistiques (`EmployeStatsDTO`)

Le calcul du solde de congé

Deux modes, contrôlés par `soldeCongeInitial` :

```
private Double soldeConge;           // solde courant
private Double soldeCongeInitial;     // saisi manuellement à la reprise de l'existant
```

« Solde congé initial saisi manuellement lors de la première utilisation de l’application. Si non null, le calcul automatique du solde est désactivé pour cet employé. »

Règle légale tunisienne implémentée (paramètres système modifiables) : - `SOLDE_CONGE_AN1 = 18` → 1,5 jour/mois × 12, la première année - `SOLDE_CONGE_AN2_PLUS = 24` → 2 jours/mois × 12, à partir de la 2^e année

`CompteService`

- Création d’un compte pour un employé + **envoi automatique des identifiants par email**
- `mustChangePassword = true` par défaut → le front force la page `/change-password`
- **Mot de passe oublié** : génération d’un `resetToken` + `resetTokenExpiry` , envoi du lien par mail, puis `POST /api/auth/reset-password`
- Changement de mot de passe (règle spéciale dans `SecurityConfig` : `PUT /api/comptes/*/password` → `authenticated` , sans besoin de `VIEW_COMPTES`)

`RoleService`

- `initPermissions()` : crée le catalogue des permissions au démarrage
- CRUD des rôles, avec affectation dynamique des permissions
- C’est ce qui rend le système **paramétrable sans redéploiement** : l’admin crée un rôle « Social Media Junior » et coche `VIEW_MEDIA_PLAN` + `VIEW_MES_PROJETS` .

7. Module Congés & Demandes

`DemandeService` — 880 lignes, le service métier le plus riche

7.1 Les trois types de demande

Type	Entité fille	Particularité
CONGE	Conge	Décompte du solde, exclusion des week-ends et fériés
AUTORISATION	Autorisation	Quota mensuel en minutes (défaut 120 min/mois)
TELETRAVAIL	Teletravail	Pas de décompte de solde
ADMINISTRATION	—	Gérée séparément via /demandes-papier (attestations, etc.)

7.2 Le calcul des jours effectifs — computeEffectiveDays()

C’est la méthode la plus délicate. Elle enchaîne :

```
1. Parcours de chaque date entre dateDebut et dateFin
2. isNonWorkingDay(date, holidays) ?
   → jour non travaillé selon HoraireTravail.joursTravail (LUNDI,MARDI,...)
   → OU jour férié présent dans la table `calendrier`
   → alors on ne le compte pas
3. Cas particulier isOnlyFridays() : une demande qui ne couvre que des vendredis
4. Retour : { joursEffectifs, joursOuvres, detail[] }
```

Le mapping jour → français est explicite car HoraireTravail.joursTravail est une chaîne "LUNDI,MARDI,MERCREDI,JEUDI,VENDREDI" :

```
private static final Map<DayOfWeek, String> DOW_TO_FRENCH = Map.of(
    DayOfWeek.MONDAY, "LUNDI", DayOfWeek.TUESDAY, "MARDI", ...);
```

7.3 Le cycle de vie d’une demande



À **chaque transition**, saveHistorique() écrit une ligne dans historique_statut (ancien statut, nouveau statut, qui, quand, commentaire) — **traçabilité complète**.

Sur approve() : décrément du solde, écriture dans le calendrier, notification. Il existe aussi batchApprove() / batchRefuse() pour traiter une liste.

7.4 Le quota d’autorisation

```
private static final int DEFAULT_MAX_AUTORISATION_MINUTES = 120; // 2 h par mois
```

Surchargeable par le référentiel MAX_AUTORISATION_MINUTES . À la création d’une autorisation, le service somme les minutes déjà consommées dans le mois et refuse au-delà du plafond.

DemandePapierService

Gère les demandes administratives (attestation de travail, attestation de salaire...) avec un flux distinct : demande → préparation → document prêt à retirer.

8. Module Calendrier & référentiels

La table des **jours particuliers de l'entreprise** : - Jours fériés (`TypeJour`) - Origine (`OrigineJour`) : légal, entreprise, exceptionnel
Ces jours sont consommés par : le calcul des congés, la paie (jours ouvrés) et l'AgentDesktop (qui ne compte pas d'absence un jour férié).

`PeriodeBloquee`

Périodes pendant lesquelles les congés sont interdits (campagne client, inventaire...). Page front : « Restrictions de congés ».

`Referentiel` — la table à tout faire

Une seule table, discriminée par `TypeReferentiel` (20 valeurs) :

```
PARAMETRE_SYSTEME · GENRE · TYPE_CONTRAT · DEPARTEMENT · POSTE
NIVEAU_HIERARCHIQUE · SITE_ETABLISSEMENT · TYPE_DEMANDE · TYPE_CONGE
FORMAT_MEDIA_PLAN · TYPE_MEDIA_PLAN · PLATFOME_MEDIA_PLAN
CATEGORIE_REVENU · CATEGORIE_CHARGE · ...
```

Avantage : l'administrateur ajoute un poste, un département, une plateforme (« TikTok »), sans redéploiement ni migration.
C'est ce qui rend l'application réellement paramétrable par le client final.

9. Module Projets, tâches, équipes

`ProjetService` (449 lignes)

CRUD projet, affectation de plusieurs chefs de projet, gestion du statut, distinction projet à durée `DETERMINE` (avec `dateFin`) / `INDETERMINE`.

`TacheService` (342 lignes)

- Création / assignation / changement de statut (`StatutTache`)
- Horodatages du cycle de vie (création, prise en charge, terminaison)
- **Génération automatique du dossier Drive** de la tâche selon `typeDrive` (Post, Video, Documentation) → `driveLink`

`ProjetAnalyseService` (516 lignes) — l'analyse du cycle de vie

C'est le module « qualité de production ». Il mesure des **délais** et lève des drapeaux :

```
/** Seuil au-delà duquel un délai de redistribution manager est signalé (24 h) */
private static final long MANAGER_DELAY_THRESHOLD_MIN = 1440L;

/** Seuil au-delà duquel un délai de démarrage membre est signalé (8 h) */
private static final long MEMBER_START_DELAY_THRESHOLD_MIN = 480L;
```

Alimente la page « **Rapport Cycle de Vie** » du frontend Projets : où le temps se perd-il entre l'affectation d'une tâche et son démarrage réel ?

`AdminDashboardService` (636 lignes)

Agrège les indicateurs pour la direction : avancement par projet, charge par employé, tâches en retard, alertes. Alimente aussi `AlerteScheduler`.

`ReactifInternService` — les « Rectifs »

Un rectif = une demande de correction sur une tâche ou une ligne de media plan. Trois types (`TypeReactif`) : `TACHE`, `MEDIA_PLAN_INTERN`, `MEDIA_PLAN_EXTERN`.

Le champ dénormalisé `nombreFois` compte **combien de fois** un même livrable a été renvoyé en correction — c'est un indicateur de qualité directement exploitable (« ce post a été corrigé 4 fois »). `dateTermineOriginal` conserve la date à laquelle la tâche avait été déclarée terminée la première fois.

`TacheObligatoireService`

Tâches récurrentes imposées à une équipe ou à un employé (ex. « rapport hebdo »). Le hook front `useTachesObligatoires` les affiche en permanence.

10. Module Media Plan & Google Drive

MediaPlanService (706 lignes)

Le media plan est **le livrable central de l'agence** : le calendrier éditorial mensuel d'une marque (quelle publication, quel jour, quelle heure, quelle plateforme, quel format, quel texte).

Fonctions : - CRUD des lignes de publication - Filtrage par client et par mois - **Périmètre** : un employé ne voit que les clients qui lui sont assignés (`media_plan_assignments`), sauf s'il a `VIEW_TOUS_MEDIA_PLAN` - Workflow de validation (`StatutMediaPlan`) : brouillon → partagé au client → validé / à corriger - Génération du **dossier Drive du mois** et remplissage de `lienDrive`

MediaPlanShootingWorkflowService (213 lignes)

Quand une ligne de media plan est marquée `isShooting = true` : 1. Un **événement de tournage** est créé dans `calendrier_projet` 2. Un **projet dédié** est généré (`isMediaPlanProject = true` , `mediaPlanLigneId`) 3. Des **tâches** sont créées pour l'équipe de production 4. Le statut du tournage (`StatutShooting`) suit son avancement

C'est le pont automatique entre « planification éditoriale » et « production ».

GoogleDriveService (311 lignes)

Utilise un **compte de service Google** (pas d'OAuth utilisateur) : - Crée l'arborescence `<Parent>/<Nom du client>/<AAAA-MM>/` - Retourne un lien partageable - Idempotent : si le dossier existe, il est réutilisé

ClientViewerDriveService (261 lignes) — le point de sécurité

```
/**
 * Read-only Drive service for the client portal.
 * Uses the "client-antigone" viewer service account.
 * Scope: DRIVE_READONLY — can list/read files but cannot create or modify anything.
 */
```

Deux comptes de service distincts :

Compte	Scope	Usage
service-account.json	Drive complet (écriture)	L'agence crée les dossiers
client-viewer-service-account.json	DRIVE_READONLY	Le portail client lit seulement

À dire en soutenance : « Le portail client utilise un compte Google distinct en lecture seule. Même une faille dans le portail ne permettrait pas à un client de modifier ou supprimer les fichiers de l'agence. C'est le principe du moindre privilège appliqué à une intégration externe. »

DriveFolderScheduler

`@Scheduled(cron = "0 0 0 1 1 ?")` → chaque 1er janvier à minuit : prépare l'arborescence Drive de la nouvelle année.

11. Module Finance — facturation

FacturationService (357 lignes)

Gère **factures et devis** (`TypeDocument.FACTURE` / `DEVIS`) dans la même entité.

La formule de tarification

```
double ht      = Math.max(sommeLignes(facture.getLignes()), nz(totalHtManuel));
double taux    = estDevis ? 0 : (tauxTva != null ? tauxTva : refDefault.getTvaDefault());
double tva     = estDevis ? 0 : ht * taux / 100.0;
double timbre  = estDevis ? 0 : (timbreFiscal != null ? timbreFiscal : refDefault.getTimbreFiscalDefault());
double ttc     = ht + tva + timbre;
```

Trois règles métier lisibles directement dans le code : 1. **Le total HT saisi manuellement agit comme un plancher, pas un remplacement** (`Math.max`) — on ne peut pas facturer moins que la somme des lignes. 2. **TVA et timbre fiscal ne s'appliquent**

jamais à un devis. 3. À défaut de valeur explicite, les taux viennent des **référentiels** (TVA 19 % et timbre fiscal tunisien paramétrables).

Numérotation

CompteurDocument garantit une numérotation **séquentielle et sans trou** par type et par année (obligation légale).

Paievements partiels

paievements_facture : plusieurs paiements par facture, montantPaye cumulé, StatutFacture recalculé (EN_ATTENTE / PARTIELLEMENT_PAYEE / PAYEE / EN_RETARD).

Relances

relances_client conserve chaque relance (objet, corps, ton, destinataire, envoyée ou non). **C’est cette table que l’IA alimente** (voir doc IA).

TvaService — la déclaration mensuelle

```
// TVA collectée sur factures : AU PRORATA DU MONTANT RÉELLEMENT ENCAISSÉ
for (Facture f : factures) {
    double proportion = Math.min(f.getMontantPaye() / f.getTotalTtc(), 1.0);
    tvaFactures += f.getMontantTva() * proportion;
}
double tvaCollectee = tvaFactures + revenuService.getTotalTvaDuMois(mois);
double tvaDeductible = chargesService.getResume(mois).getTvaDeductible();
double tvaNette = tvaCollectee - tvaDeductible;
```

Le détail qui montre la maîtrise métier : la TVA n’est due que sur ce qui est **encaissé**, pas sur ce qui est facturé. Une facture payée à 40 % ne génère que 40 % de sa TVA. C’est la règle de l’encaissement, et elle est implémentée.

ChargesService / DetteService / RevenuService

Service	Périmètre
ChargesService	Charges fixes (loyer, abonnements) avec leurs paiements + charges variables
DetteService	Dettes de l’agence : soldeRestant = max(0, montantTotal - montantPaye)
RevenuService	Revenus hors facturation. Le montant saisi est TTC, le HT en est extrait

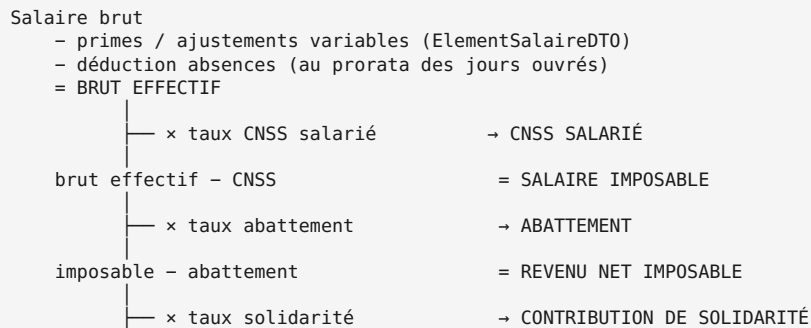
DashboardFinanceService

Trois vues consolidées : **encaissements, décaissements, résultat net** (VueEncaissementsDT0 , VueDecaissementsDT0 , ResultatNetDT0).

12. Module Paie tunisienne

C’est la partie la plus technique du backend. Elle est isolée dans PayrollCalculator (288 lignes, @Component pur, sans base de données) et orchestrée par PayrollService (306 lignes).

12.1 La chaîne de calcul du net



```

    |— barème IRPP progressif → IRPP MENSUEL
    |   irppAnnuel(RNI × 12) / 12
brut effectif - CNSS - solidarité - IRPP = NET
    |
    |— acomptes déjà versés
    = NET À PAYER

```

Côté employeur, en parallèle : `cnssEmployeurDetail`, `tfpDetail` (taxe de formation professionnelle), `foprolosDetail` (fonds de promotion du logement), `atDetail` (accidents du travail) → `chargesEmployeur` → `coutTotal`.

12.2 Le barème IRPP progressif

```

public double irppAnnuel(double imposableAnnuel, List<TrancheIrppDTO> tranches) {
    double impot = 0, reste = imposableAnnuel, prev = 0;
    for (TrancheIrppDTO t : tranches) {
        double plafond = t.getPlafond() == null ? Double.POSITIVE_INFINITY : t.getPlafond();
        double part = Math.min(reste, plafond - prev);
        if (part <= 0) break;
        impot += part * t.getTaux();
        reste -= part;
        prev = plafond;
    }
    return impot;
}

```

Chaque tranche est imposée à son propre taux — **ce n'est pas un taux unique appliqué au revenu total**. Erreur classique évitée.

12.3 L'inversion NET → BRUT par dichotomie ☆

Le morceau à montrer au jury. En Tunisie, on négocie souvent un salaire **net**. Or le calcul va du brut vers le net, et l'IRPP étant **progressif par tranches**, la fonction n'est pas inversible analytiquement.

```

/**
 * Recherche par dichotomie du salaire brut tel que le net calculé égale le net cible.
 * L'IRPP étant progressif, cette relation ne peut pas être inversée analytiquement.
 */
public double brutDepuisNet(double netCible, ParametresPaieDTO taux, List<TrancheIrppDTO> tranches) {
    double low = netCible; // le brut est forcément ≥ au net
    double high = netCible * 3; // borne haute large

    for (int i = 0; i < 64; i++) { // 64 itérations = précision machine
        double mid = (low + high) / 2;
        // ... on recalcule tout le net à partir de `mid` ...
        if (Math.abs(netCalcule - netCible) < 0.0005) break; // convergence à 0,05 millime
        if (netCalcule < netCible) low = mid; else high = mid;
    }
    return round3((low + high) / 2);
}

```

L'employé porte un champ `modeSalaire` = "BRUT" ou "NET" : si **NET**, le système lance la dichotomie pour retrouver le brut correspondant.

Phrase de soutenance : « La conversion net → brut est un problème d'inversion de fonction non linéaire. J'ai utilisé une recherche dichotomique sur 64 itérations, avec une convergence à 0,0005 dinar. »

12.4 Les cas de bord gérés

Cas	Méthode	Traitement
Embauche en cours de mois	<code>joursTravaillésPremierMois()</code>	Prorata sur les jours ouvrés depuis la date d'embauche
Départ en cours de mois	<code>joursTravaillésDernierMois()</code>	Prorata jusqu'à <code>archivedAt</code>
Employé actif ce mois-ci ?	<code>estActifPourMois()</code>	Croise embauche / fin de contrat / archivage
Contrat exonéré	<code>estExonere()</code>	CIVP, Freelance, Stage → aucune retenue CNSS/IRPP/solidarité

```

private static final Set<String> TYPES_EXONERES = Set.of("CIVP", "Freelance", "Stage");

public boolean estExonere(String typeContrat) {
    // Le test de nullité est indispensable : Set.of(...).contains(null) lève une NPE.
    return typeContrat != null && TYPES_EXONERES.contains(typeContrat);
}

```

12.5 CnssService — la déclaration trimestrielle

```
// Somme des 3 bulletins du trimestre
montantSalarie += b.getCnssSalarie() + b.getSolidariteSalarie();
montantEmployeur += b.getCnssEmployeurDetail();

// Échéance légale : le 15 du mois suivant la fin du trimestre
public LocalDate echeanceTrimestre(int annee, int trimestre) {
    return LocalDate.of(annee, trimestre * 3, 15).plusMonths(1);
}
```

Plus le calcul automatique de la **pénalité de retard** si la déclaration est déposée après l’échéance.

12.6 BulletinPaie — toutes les étapes persistées

L’entité stocke **chaque étape intermédiaire**, pas seulement le résultat :

```
salaireBrut · bonus · deductionAbsences · brutEffectif · cnssSalarie ·
salaireImposable · abattementMontant · revenuNetImposable · irppMensuel ·
solidariteSalarie · net · acompte · netAPayer ·
chargesEmployeur · coutTotal · cnssEmployeurDetail · tfpDetail ·
foprolosDetail · atDetail
```

Pourquoi ? Un bulletin de paie doit être **justifiable et reproductible** même si les taux changent l’année suivante. Et c’est exactement ce qui permet à l’IA d’expliquer un bulletin ligne par ligne **sans jamais recalculer**.

13. Module Monitoring & AgentDesktop

13.1 L’application AgentDesktop (Electron 28)

Installée sur le poste de chaque employé. Fichiers dans AgentDesktop/src/ :

Fichier	Rôle
main.js	Processus principal, verrou d’instance unique, gestion du cycle de vie
login.html	Connexion avec les mêmes identifiants que les apps web
activity.js	Détection d’activité via powerMonitor.getIdleTime()
network.js	Récupération de l’IP locale et du SSID Wi-Fi
heartbeat.js	Envoi périodique de l’état au backend
popup-manager.js	Popups de confirmation de présence / validation de fin de journée
tray.js	Icône dans la barre système
config.js	Récupération de la configuration horaire depuis le backend

Paramètres clés :

```
const INACTIVITY_THRESHOLD_MS = 120000; // 2 min sans activité = inactif
setInterval(checkActivity, 30000); // vérification toutes les 30 s
setInterval(sendHeartbeat, 60000); // heartbeat toutes les 60 s
setInterval(refreshConfig, 3600000); // config rafraîchie 1x/h (1x/jour effectif)
```

Principe d’architecture affiché dans le code :

```
// L'agent collecte et envoie les données brutes (IP, SSID, activité)
// Le BACKEND gère toute la logique métier (vérification SSID, horaires, retard, etc.)
```

Excellent argument de soutenance : « L’agent est volontairement stupide. Il ne décide de rien. Toute la logique est côté serveur — donc modifiable sans réinstaller l’agent sur 30 postes, et non contournable en modifiant le client. »

13.2 Côté backend : AgentService (573 lignes)

Endpoints consommés par l’agent : - GET /api/agent/config → horaires, jours fériés de l’année, jours travaillés - POST

`/api/agent/heartbeat` → réception de l'état (IP, SSID, actif/inactif) - `POST /api/agent/presence-confirm` → réponse de l'employé à une popup - `GET /api/agent/download` (public) → téléchargement de l'installateur

`AgentService` croise le heartbeat avec : - l'horaire de travail affecté (`AffectationHoraire` → `HoraireTravail`) - les jours fériés (`Calendrier`) - **les congés, autorisations et télétravail approuvés** — pour ne pas compter une absence justifiée comme un retard

13.3 `PointageScheduledService` — la fermeture automatique

```
private static final int SEUIL_INACTIVITE_MINUTES = 10;

@Scheduled(fixedDelay = 5 * 60 * 1000) // toutes les 5 minutes
public void autoFillHeureSortie() {
    // Pour chaque pointage du jour sans heureSortie :
    // si le dernier heartbeat date de plus de 10 minutes
    // → heureSortie = timestamp du dernier heartbeat
}
```

Le problème résolu : un employé éteint son PC sans se déconnecter. Sans ce scheduler, son pointage resterait ouvert indéfiniment. On ne prend pas l'heure courante mais **l'heure du dernier heartbeat** — c'est l'heure réelle de départ.

13.4 `RapportInactiviteService`

Génère des rapports d'inactivité soumis à **décision managériale** (`RapportDecisionRequest`) : le manager confirme ou justifie l'inactivité détectée. L'automatisme ne sanctionne jamais seul.

14. Notifications et emails

`NotificationService` — notifications in-app

```
create(employe, titre, message, demande) // liée à une demande
createUrgent(employe, titre, message, demande) // marquée urgente
createSimple(employe, titre, message) // libre
createForReunion(employe, titre, message, reunion)
getUnreadByEmploye(id) · countUnread(id) · markAsRead(id) · markAllAsRead(id)
```

Côté frontend : `NotificationContext` + un poller, avec `utils/notificationRules.ts` pour le routage et l'affichage.

`EmailService` — double canal, avec repli

```
public void sendHtml(String toEmail, String subject, String htmlBody, String recipientName) {
    if (!brevoApiKey.isBlank()) {
        sendViaBrevoApi(...); // API HTTP Brevo
    } else {
        ... // repli sur SMTP (JavaMailSender)
    }
}
```

Pourquoi Brevo en priorité ? En production sur Render, le port SMTP sortant est souvent bloqué et les envois Gmail finissent en spam. Brevo passe en **HTTPS sur le port 443** et gère la réputation d'envoi. Le SMTP reste comme repli local.

Usages : envoi des identifiants (`sendCredentials`), réinitialisation de mot de passe (`sendPasswordReset`), et **relances client générées par l'IA**.

Détail soigné : le logo du client dans l'email de relance est servi par l'endpoint public `GET /api/clients/{id}/logo`, construit avec `app.public-backend-url` :

« Un client mail charge une image par URL absolue, jamais par un chemin relatif. »

15. Les tâches planifiées (schedulers)

Classe	Cron	Rôle
<code>AlerteScheduler</code>	<code>0 0 */2 * * *</code> (toutes les 2 h)	Alertes tâches en dépassement pour admin et managers

TacheDeadlineScheduler	0 0 8 * * * (8 h)	Rappels d'échéance de tâches
TacheArchivingScheduler	0 0 * * * * (toutes les heures)	Archivage des tâches terminées anciennes
DriveFolderScheduler	0 0 0 1 1 ? (1er janvier)	Création de l'arborescence Drive annuelle
ContratExpirationScheduler	0 0 8 * * * (8 h)	Alerte sur les contrats arrivant à échéance
PointageScheduledService	fixedDelay = 5 min	Fermeture auto des pointages
EmbeddingIndexService (IA)	0 0 3 * * * (3 h)	Réindexation du RAG

16. Gestion des erreurs

GlobalExceptionHandler (@RestControllerAdvice)

Traduit les exceptions en réponses HTTP cohérentes, pour que le frontend n'ait jamais à interpréter une stack trace.

ApiResponse<T> — enveloppe uniforme

```
{ "success": true, "message": "...", "data": { ... } }
```

Tous les contrôleurs métier renvoient cette enveloppe → le frontend a **un seul format à gérer**.

Exceptions du module IA

AiForbiddenException (403), AiRateLimitException (429), AiUnavailableException (503), traitées par AiExceptionHandler — un @RestControllerAdvice **distinct**, pour que les codes IA ne polluent pas la sémantique du métier.

17. Configuration et environnements

application.yml — les blocs

```
server.port: ${PORT:8080}                # Render impose PORT
app.frontend-url: ${FRONTEND_URL:...}      # CORS, plusieurs origines séparées par virgule
app.public-backend-url: ...               # liens absolus dans les emails
app.jwt.secret / expiration-hours: 12
app.mail.brevo-api-key / from
app.google.drive.*                        # 2 clés de service, parent-folder-id
app.ai.*                                  # tout le module IA (voir doc IA)
spring.datasource.*                       # PostgreSQL
spring.jpa.hibernate.ddl-auto: update
spring.sql.init.mode: always              # exécute data.sql
spring.mvc.async.request-timeout: -1      # Δ indispensable au SSE
spring.servlet.multipart.max-file-size: -1 # pas de limite d'upload
```

Deux lignes critiques pour l'IA :

```
spring.mvc.async.request-timeout: -1
# -1 = le conteneur n'interrompt jamais une requête asynchrone.
# Sans cela, Spring MVC coupe les flux SSE après son délai par défaut.
```

Profiles

- application.yml → développement (Postgres local)
- application-prod.yml → production (SPRING_PROFILES_ACTIVE=prod sur Render)

⚠ Point de vigilance à corriger avant la soutenance

Le fichier Backend/src/main/resources/application.yml contient actuellement une **clé API OpenAI écrite en dur** (ligne api-key: sk-proj-...), au lieu de \${OPENAI_API_KEY:}. Comme le fichier est versionné dans Git, cette clé est **exposée dans l'historique du dépôt**.

À faire : 1. Révoquer la clé sur la console OpenAI (elle doit être considérée comme compromise). 2. Générer une nouvelle clé. 3. Remplacer la ligne par api-key: \${OPENAI_API_KEY:} — le reste du fichier utilise déjà ce pattern partout ailleurs, donc c'est une correction d'une ligne. 4. Définir OPENAI_API_KEY en variable d'environnement (elle est déjà déclarée dans render.yaml avec sync: false).

C’est aussi **un très bon point à mentionner spontanément** en soutenance : montrer qu’on sait identifier une fuite de secret et qu’on connaît la procédure de remédiation.

18. Tests

```
<!-- surefire (mvn test) → *Test : unitaires, sans Docker -->
<!-- failsafe (mvn verify) → *IT : intégration, Testcontainers + Postgres réel -->
```

Pourquoi Testcontainers plutôt que H2 ? Le projet utilise des fonctionnalités spécifiques à PostgreSQL : recherche plein texte `to_tsvector('french', ...)`, extension `pgvector`, requêtes natives. H2 ne les supporte pas — un test qui passe sur H2 ne prouverait rien.

Le widget IA a sa propre pile : **Vitest** (unitaire), **Testing Library**, **MSW** (mock des appels HTTP) et **Playwright** (end-to-end).

19. Le module IA

Le package `com.antigone.rh.ai` compte 5 contrôleurs, ~10 services, 6 classes d’outils, un moteur RAG hybride, une mémoire conversationnelle et un système SSE.

📄 **Voir le fichier** `05-IA-EXPLIQUEE-SIMPLEMENT.md`, entièrement consacré à cette partie.

Annexe A — Les 35 contrôleurs REST

Contrôleur	Base URL	Domaine
AuthController	/api/auth	Login employé, login client, mot de passe oublié
EmployeController	/api/employes	CRUD employés, organigramme, archives
CompteController	/api/comptes	Comptes, mots de passe
RoleController	/api/roles	Rôles et permissions
CompetenceController	/api/competences	Compétences
DocumentEmployeController	/api/documents-employe	Documents RH
DemandeController	/api/demandes	Congés, autorisations, télétravail
DemandePapierController	/api/demandes-papier	Demandes administratives
ValidationController	/api/validations	Validation des demandes
CalendrierController	/api/calendrier	Jours fériés, périodes bloquées
ReferentielController	/api/referentiels	Référentiels paramétrables
EquipeController	/api/equipes	Équipes
ProjetController	/api/projets	Projets
TacheController	/api/taches	Tâches
TacheObligatoireController	/api/taches-obligatoires	Tâches récurrentes
ProjetAnalyseController	/api/projets/analyse	Analyse cycle de vie
AdminDashboardController	/api/admin/dashboard	Tableau de bord direction
CalendrierProjetController	/api/calendrier-projet	Deadlines, réunions, tournages
ReunionController	/api/reunions	Réunions
ClientController	/api/clients	Clients, logo, portail
ContactClientController	/api/contacts-client	Contacts clients
MediaPlanController	/api/media-plans	Media plans + callback Drive
MediaPlanAssignmentController	/api/media-plan-assignments	Affectation employé↔client

MediaPlanCommentController	/api/media-plan-comments	Commentaires
ReactifInternController	/api/reactifs	Rectifs
FacturationController	/api/finance/factures	Factures & devis
RevenuController	/api/finance/revenus	Autres revenus
ChargesController	/api/finance/charges	Charges fixes et variables
DetteController	/api/finance/dettes	Dettes
PayrollController	/api/finance/paie	Bulletins de paie
CnssController	/api/finance/cnss	Déclarations CNSS
FinanceDashboardController	/api/finance/dashboard	Encaissements / décaissements
FinanceReferentielController	/api/finance/referentiels	Taux TVA, timbre fiscal
AgentController	/api/agent	AgentDesktop
NotificationController	/api/notifications	Notifications

Plus les 5 contrôleurs IA sous /api/v1/ .

Annexe B — Documentation interactive

Swagger UI est actif et **public** :

```
http://localhost:8080/swagger-ui.html
```

🔑 **Ouvrez-le pendant la démonstration** : c’est la façon la plus rapide de montrer au jury l’étendue de l’API sans faire défiler du code.